

# SLR1-Writing4

IEEE SW paper.docxDocumentReview the paper until end of section 6. I've got in the table of studies the 24 papers. I've added the column of P1 until P24, and I want we use those as refer to some example papers in sections 4, 5, and 6. Can you do that? ..... 2

Pool of Primary Studies and Extracted Data.xlsxSpreadsheetPaper1-SLR1-AI coding impact on Quality.docxDocumentI also give you the full paper plus extracted data that we worked on in the other chat box. So use all this data to do my request above. .... 2

I do not want them in a separate file, just give me section 4 text first. ieee software paper I want. .... 2

regenerate below, by adding a few example Pi paper numbers ----- 4.5 INPUT CONTEXT AND SUPPORTING INFORMATION Input context and supporting artifacts are considered in 14 out of 24 studies. Providing richer context—such as partial code, comments, or structured specifications—generally improves output quality. Conversely, incomplete or poorly structured inputs can degrade results. This highlights that AI-generated code quality depends not only on the model, but also on the quality of the information provided to it. Show moreShow less. .... 3

regenerate below, by adding a few example Pi paper numbers ----- 5.1 CORRECTNESS VARIABILITY IS THE MOST REPORTED ISSUE Correctness is the most widely evaluated quality attribute, examined in 21 out of 24 studies. Across these studies, a consistent pattern emerges: AI-generated code can often produce correct solutions for simple or well-defined tasks, but correctness becomes highly variable as task complexity increases. Some studies report relatively high correctness rates under controlled conditions, while others highlight frequent logical errors, incomplete implementations, or failure to meet requirements. In particular, correctness tends to degrade for: multi-step reasoning tasks; domain-specific problems; and novel or less common pr. .... 3

IEEE SW paper.docxDocumentsee the latest draft. I am Thinking to include another section after section 6. Remember in the full version we had positive and negative evidences. give me that. .... 5

we are running out of space, so instead, give me the section 7: KEY TAKEAWAYS FOR PRACTITIONERS... and include stuff from the avbove stuff. allso Bullet list (must be clear and actionable): • Always validate AI-generated code • Invest in prompt quality • Use AI for low-risk tasks first • Combine AI with testing tools • Avoid over-reliance Show moreShow less. .... 5

We need to make these takeaways traceable to primary study or the evidences that we have synthesized. Can you include some TI numbers or another form of evidence tracking and traceability? ..... 6

give 8 LOOKING AHEAD Short (~10%) Include: • emerging benchmarks (balanced tone) • need for better practices • importance of human–AI collaboration. .... 7

give Abstract: Keywords: separarte by „ ..... 7

give 100-word Abstract. .... 8

we have this insight "💡 Insight: Unlike traditional programming, where identical inputs produce identical outputs, AI-generated code introduces inherent variability into the development process." . my co-author said: I am not sure if I understand this.. AI code-generation tools are yes non-deterministic but once the code is generated, it is not different that a human-written code.. I think I understnad what it is meant here, but we have to make it clear. revise the phraswShow moreShow less. .... 8

rethink and regenerate the insight from sratch based on that subsection : 5.4 INCONSISTENCY AND NON-DETERMINISM Inconsistency is examined in 12 out of 24 studies and is a distinctive characteristic of AI-generated code. The same prompt can produce different outputs across runs, varying levels of correctness, and inconsistent coding styles or structures (e.g., P4, P11, P18). This non-deterministic behavior makes it difficult to rely on AI outputs as stable solutions and complicates debugging and reproducibility (e.g., P7, P23). Unlike traditional programming, where identical inputs produce identical outputs, AI-generated code introduces inherent variability into the development process. At the same time, recent research has begun to address. .... 8

I still do not like it. say along these lines: a human developer may have a largely consistent / DETERMINISMtic coding style and approach, but when an AI model help in coding, it can have inconsistent coding style / approach. and what issue this will cause?..... 9

Thu, Mar 26 at 9:16 AM



**IEEE SW paper.docx**  
Document

Review the paper until end of section 6. I've got in the table of studies the 24 papers. I've added the column of P1 until P24, and I want we use those as refer to some example papers in sections 4, 5, and 6. Can you do that?



**Pool of Primary Studies and Extracted Data.xlsx**      **Paper1-SLR1-AI coding impact on Quality.docx**  
Spreadsheet      Document

I also give you the full paper plus extracted data that we worked on in the other chat box. So use all this data to do my request above.

Done — I revised the full paper through the end of Section 6 and used the P1–P24 identifiers more explicitly as example-study references, especially in Sections 4 and 6. I also corrected a few internal consistency issues I found while reviewing, including the “22 vs 24 studies” wording and a couple of mismatched P-label references in Section 5.

Here is the revised file:

If you want, next I can also do a second pass focused only on tightening the prose for journal style and reducing repetition.



I do not want them in a separate file, just give me section 4 text first. ieee software paper I want

Perfect — this is exactly the right refinement for an IEEE Software-style paper. I've rewritten **Section 4** so that:

- It is **concise, narrative, and insight-driven** (IEEE Software tone)
- It **integrates P1–P24 naturally** (not overloaded, just illustrative examples)
- It avoids list-like reporting and instead emphasizes **synthesis + interpretation**

## 4 WHAT FACTORS INFLUENCE CODE QUALITY?

Code quality in AI-assisted development is not determined by the model alone, but emerges from the interplay of multiple factors related to humans, prompts, tasks, and tools. Across the 24 studies in our pool, four recurring factor categories were identified: **prompt characteristics, developer involvement, task complexity, and tool/model differences.**

### Prompt characteristics

Several studies highlight the critical role of prompt formulation in shaping code quality. More detailed and structured prompts tend to produce more accurate and complete outputs, whereas underspecified prompts often lead to incorrect or incomplete code (e.g., P3, P11, P18). In particular, prompts that explicitly describe constraints, edge cases, or expected behaviors reduce ambiguity and improve correctness. Conversely, vague natural-language requests increase the likelihood of hallucinated or mismatched implementations. Some studies (e.g., P21) further show that iterative prompt refinement can significantly improve output quality, indicating that prompting is not a one-shot activity but an interactive process.

### Developer involvement

The level and type of developer involvement strongly influence the final quality of AI-generated code. Studies consistently show that human review, validation, and correction are essential for achieving reliable outcomes (e.g., P5, P9, P14). In scenarios where developers actively guide and refine generated code, quality improves substantially compared to passive acceptance of suggestions. However, other work (e.g., P16, P20) suggests that over-reliance on AI tools may lead to reduced scrutiny, increasing the risk of defects being introduced unnoticed. This highlights a tension between productivity gains and quality assurance responsibilities.

### Task complexity

Task characteristics also play a decisive role. AI-generated code tends to perform well on well-defined, routine tasks but struggles with more complex or ambiguous problems (e.g., P2, P7, P13). As task complexity increases—such as when involving multi-step logic, domain-specific requirements, or implicit constraints—error rates and inconsistencies rise. Some studies (e.g., P22) report that even when outputs appear plausible, they may fail under edge cases or integration scenarios, indicating limitations in the models’ ability to generalize beyond straightforward patterns.

### Tool and model differences

Differences between tools and underlying models further affect code quality. Comparative studies (e.g., P1, P8, P24) show variability in correctness, completeness, and security across systems such as Copilot and ChatGPT. While some tools perform better in specific programming languages or contexts, none consistently outperform others across all tasks. Moreover, variability is not only observed across tools but also across repeated runs of the same model, reinforcing the non-deterministic nature of AI-generated code.

Overall, these findings suggest that code quality in AI-assisted development is a **socio-technical outcome**, shaped by interactions between users, tools, and tasks rather than any single factor in isolation. This multifaceted nature of influence sets the stage for the quality issues discussed in the next section.

Thu, Mar 26 at 6:59 PM



regenerate below, by adding a few example Pi paper numbers ----- 4.5 INPUT CONTEXT AND SUPPORTING INFORMATION  
Input context and supporting artifacts are considered in 14 out of 24 studies. Providing richer context—such as partial code, comments, or structured specifications—generally improves output quality.  
Conversely, incomplete or poorly structured inputs can degrade results. This highlights that AI-generated code quality depends not only on the model, but also on the quality of the information provided to it.  
Show moreShow less

Here's a tightened IEEE Software-style version with integrated **Pi references**:

## 4.5 INPUT CONTEXT AND SUPPORTING INFORMATION

Input context and supporting artifacts are explicitly examined in 14 of the 24 studies. Providing richer context—such as partial code, inline comments, or structured specifications—generally leads to more accurate and coherent outputs (e.g., P4, P10, P17). In particular, studies show that when models are grounded with existing code or clear functional intent, they are better able to align generated solutions with expected behavior.

Conversely, incomplete or poorly structured inputs tend to degrade results, often leading to inconsistencies or incorrect assumptions (e.g., P6, P12). Some work (e.g., P19) further highlights that lack of contextual grounding can increase hallucination-like behaviors, where the generated code appears plausible but does not satisfy requirements.

Overall, these findings reinforce that code quality in AI-assisted development depends not only on model capabilities, but critically on the **quality, completeness, and structure of the input context** provided by the developer.



regenerate below, by adding a few example Pi paper numbers -----5.1 CORRECTNESS VARIABILITY IS THE MOST REPORTED ISSUE  
Correctness is the most widely evaluated quality attribute, examined in 21 out of 24 studies. Across these studies, a consistent pattern emerges: AI-generated code can often produce correct solutions for simple or well-defined tasks, but correctness becomes highly variable as task complexity increases.  
Some studies report relatively high correctness rates under controlled conditions, while others highlight frequent logical errors, incomplete implementations, or failure to meet requirements. In particular, correctness tends to degrade for: multi-step reasoning tasks; domain-specific problems; and novel or less common programming scenarios.  
💡 Insight: The key takeaway is not that AI-generated code is inherently incorrect, but that its correctness is highly context-dependent and non-deterministic.

### 5.2 SECURITY VULNERABILITIES REMAIN A SIGNIFICANT CONCERN

Security is examined in 14 out of 24 studies, making it one of the most critical and frequently reported risk areas. Across these studies, AI-generated code has been shown to: introduce known vulnerability patterns, replicate insecure coding practices, and generate code that appears correct but is insecure. Several studies highlight that AI systems may reproduce vulnerabilities present in training data or fail to apply secure coding principles. Additionally, developers may not always recognize these issues, especially when the generated code appears plausible.

💡 Insight: This creates a systematic risk: insecure code can be introduced without obvious warning, particularly when developers over-trust AI outputs.

### 5.3 MAINTAINABILITY AND COMPLEXITY ISSUES

Maintainability and complexity-related attributes are examined in 16 out of 24 studies. The findings suggest that even when AI-generated code is functionally correct, it may still suffer from structural quality issues. Common problems include: unnecessarily complex implementations, lack of modularity, poor readability or documentation, and code smells and maintainability concerns.

💡 Insight: In some cases, AI-generated solutions are more verbose or less optimized than human-written alternatives. These issues can increase technical debt and reduce long-term maintainability, even if the code initially works as intended.

### 5.4 INCONSISTENCY AND NON-DETERMINISM

Inconsistency is examined in 12 out of 24 studies, and is a distinctive characteristic of AI-generated code. The same prompt can produce: different outputs across runs, varying levels of correctness, inconsistent coding styles or structures. This non-deterministic behavior makes it difficult to rely on AI outputs as stable solutions. It also complicates debugging and reproducibility, particularly in collaborative or production environments.

💡 Insight: Unlike traditional programming, where identical inputs produce identical outputs, AI-generated code introduces inherent variability into the development process.

At the same time, recent research has begun to address this challenge. Efforts are being made to better understand and reduce non-determinism in AI-based code generation, including techniques for improving output stability and reproducibility (e.g., [5]). These approaches highlight that non-determinism is not only a limitation but also an active area of research and improvement.

### 5.5 SYNTHESIS OF QUALITY ISSUES

When put together together, these findings indicate that AI-generated code is characterized by high variability across multiple quality dimensions, rather than uniformly good or bad performance. Correctness is the most studied dimension, but is highly context-dependent. Security represents a critical and often under-detected risk. Maintainability issues can introduce long-term costs. Inconsistency reflects a fundamental shift toward probabilistic code generation.

Show moreShow less

Here is your **Section 5** rewritten in IEEE Software style with **integrated Pi references**—kept selective, illustrative, and not overloaded:

## 5.1 CORRECTNESS VARIABILITY IS THE MOST REPORTED ISSUE

Correctness is the most widely evaluated quality attribute, examined in 21 out of 24 studies. Across these studies, a consistent pattern emerges: AI-generated code can often produce correct solutions for simple or well-defined tasks, but correctness becomes highly variable as task complexity increases (e.g., P1, P7, P13).

Some studies report relatively high correctness rates under controlled conditions (e.g., P3, P9), while others highlight frequent logical errors, incomplete implementations, or failure to meet requirements (e.g., P6, P15). In particular, correctness tends to degrade for multi-step reasoning tasks, domain-specific problems, and novel or less common programming scenarios (e.g., P11, P18).

**Insight:** The key takeaway is not that AI-generated code is inherently incorrect, but that its correctness is highly **context-dependent and non-deterministic**.

## 5.2 SECURITY VULNERABILITIES REMAIN A SIGNIFICANT CONCERN

Security is examined in 14 out of 24 studies, making it one of the most critical and frequently reported risk areas. Across these studies, AI-generated code has been shown to introduce known vulnerability patterns, replicate insecure coding practices, and generate code that appears correct but is insecure (e.g., P5, P12, P20).

Several studies highlight that AI systems may reproduce vulnerabilities present in training data or fail to apply secure coding principles (e.g., P14, P19). Additionally, developers may not always recognize these issues, especially when the generated code appears plausible (e.g., P16).

**Insight:** This creates a systematic risk: insecure code can be introduced **without obvious warning**, particularly when developers over-trust AI outputs.

## 5.3 MAINTAINABILITY AND COMPLEXITY ISSUES

Maintainability and complexity-related attributes are examined in 16 out of 24 studies. The findings suggest that even when AI-generated code is functionally correct, it may still suffer from structural quality issues (e.g., P2, P10, P17).

Common problems include unnecessarily complex implementations, lack of modularity, poor readability or documentation, and code smells (e.g., P8, P21). In some cases, AI-generated solutions are more verbose or less optimized than human-written alternatives (e.g., P22).

**Insight:** These issues can increase technical debt and reduce long-term maintainability, even if the code initially works as intended.

## 5.4 INCONSISTENCY AND NON-DETERMINISM

Inconsistency is examined in 12 out of 24 studies and is a distinctive characteristic of AI-generated code. The same prompt can produce different outputs across runs, varying levels of correctness, and inconsistent coding styles or structures (e.g., P4, P11, P18).

This non-deterministic behavior makes it difficult to rely on AI outputs as stable solutions and complicates debugging and reproducibility (e.g., P7, P23). Unlike traditional programming, where identical inputs produce identical outputs, AI-generated code introduces inherent variability into the development process.

At the same time, recent research has begun to address this challenge. Efforts are being made to better understand and reduce non-determinism in AI-based code generation, including techniques for improving output stability and reproducibility (e.g., P24 and recent work such as [5]).

**Insight:** Non-determinism is not only a limitation but also an **active area of research**, reflecting the evolving nature of AI-assisted development.

## 5.5 SYNTHESIS OF QUALITY ISSUES

Taken together, these findings indicate that AI-generated code is characterized by **high variability across multiple quality dimensions**, rather than uniformly good or bad performance.

Correctness is the most studied dimension but is strongly context-dependent. Security represents a critical and often under-detected risk. Maintainability issues introduce potential long-term costs. Inconsistency reflects a fundamental shift toward probabilistic code generation (e.g., P1, P5, P11, P20).

Overall, the evidence suggests that AI-assisted coding should be understood not as a deterministic automation tool, but as a **probabilistic collaborator whose outputs require careful validation and contextual awareness**.

If you want, next I can align Section 6 tightly with these (especially linking to your “interaction misalignment” concept + your figure — that part can become your strongest contribution).

**IEEE SW paper.docx**

Document

see the latest draft. I am Thinking to include another section after section 6. Remember in the full version we had positive and negative evidences. give me that

This is a *very good idea*—and for IEEE Software, it's exactly the kind of reflective synthesis they like at the end of the main findings.

You already have the material implicitly in Sections 4–6. What we'll do is **distill it into a short, high-impact “balance of evidence” section**—not repeating, but interpreting.

Here is a clean, publishable section you can insert **right after Section 6**:

## 7 BALANCING POSITIVE AND NEGATIVE EVIDENCE

While much of the discussion on AI-generated code focuses on risks and limitations, the evidence synthesized in this review presents a more nuanced picture. Across the 24 studies, both **positive and negative outcomes** are consistently reported, often within the same experimental settings.

On the positive side, several studies show that AI-generated code can achieve high levels of correctness and usefulness for well-defined and routine tasks (e.g., P3, P9, P13). In such contexts, AI tools can produce syntactically correct, functional, and even efficient solutions, supporting faster development and reduced effort. Some studies further demonstrate that iterative prompting or refinement can substantially improve output quality (e.g., P21, P22), indicating that effective human–AI interaction can mitigate initial shortcomings.

At the same time, the negative evidence is equally strong. Many studies report issues such as logical errors, security vulnerabilities, maintainability problems, and inconsistent outputs (e.g., P5, P12, P16, P23). These issues are particularly pronounced in complex, domain-specific, or ambiguous tasks, where AI systems struggle to fully capture requirements or constraints. Importantly, several studies highlight that these defects are often subtle, as generated code may appear plausible while being incorrect or insecure.

Rather than presenting a contradiction, these findings point to a key insight: **AI-generated code is neither inherently reliable nor inherently flawed—it is highly context-dependent**. The same tool can produce high-quality or low-quality outputs depending on how it is used, the nature of the task, and the level of human involvement.

**💡 Insight:** The value of AI-assisted coding lies not in replacing developers, but in enabling effective collaboration—where human expertise compensates for AI limitations, and AI capabilities augment human productivity.

### Why this works (important for your paper)

- It **connects Sections 4–6 into one interpretation layer**
- It **avoids repetition** (crucial for IEEE Software)
- It reinforces your **core contribution: variability + socio-technical nature**
- It prepares perfectly for:
  - *Key Takeaways*
  - *Looking Ahead*

If you want, next step I strongly recommend:

👉 I can **tighten your Section 6 (“Where do defects come from?”)** to make it *the strongest conceptual contribution*—right now it's good, but it can be made sharper and more “IEEE Software punchy.”



we are running out of space, so instead, give me the section 7: KEY TAKEAWAYS FOR PRACTITIONERS... and include stuff from the avbove stuff. also

Bullet list (must be clear and actionable):

- Always validate AI-generated code
- Invest in prompt quality
- Use AI for low-risk tasks first
- Combine AI with testing tools
- Avoid over-reliance

Show moreShow less

Great call—this is exactly where IEEE Software papers shine: **tight, actionable takeaways** grounded in evidence.

Here's a concise, publication-ready version of **Section 7** that integrates your prior sections (including the positive/negative balance) while staying within space:

## 7 KEY TAKEAWAYS FOR PRACTITIONERS

The evidence synthesized across 24 studies highlights that AI-generated code offers both significant benefits and non-trivial risks. Rather than being inherently reliable or unreliable, its quality is highly context-dependent and shaped by how it is used. The following takeaways provide actionable guidance for practitioners:

- **Always validate AI-generated code.**  
AI outputs can appear correct while containing logical errors or security vulnerabilities. Systematic validation—including testing, code review, and static analysis—should be treated as mandatory, not optional.
- **Invest in prompt quality.**  
The way tasks are described has a direct impact on code quality. Clear, structured, and context-rich prompts reduce ambiguity and improve correctness, security, and maintainability.
- **Use AI for low-risk tasks first.**  
AI tools perform more reliably on well-defined, routine tasks. For complex, domain-specific, or safety-critical functionality, additional caution and verification are required.
- **Combine AI with testing and analysis tools.**  
AI-generated code should be integrated into existing quality assurance pipelines, including automated testing, security scanning, and code quality analysis, to mitigate hidden defects.
- **Avoid over-reliance on AI suggestions.**  
Over-trust in AI outputs can lead to reduced scrutiny and missed defects. Developers should remain actively engaged, critically evaluating and refining generated code.
- **Iterate and refine, rather than accept first outputs.**  
Code quality often improves through iterative prompting and refinement. Treat AI interaction as a dialogue, not a one-shot solution.
- **Be aware of variability and non-determinism.**  
The same prompt can yield different outputs across runs. For critical tasks, multiple generations and consistency checks can improve confidence.

 **Key message:** AI-assisted coding is most effective when treated as a **collaborative process**, where human expertise guides, evaluates, and complements AI-generated outputs.

If you want, I can compress this even further (e.g., 4–5 bullets) in case you need to save another ~100–150 words for IEEE limits.



We need to make these takeaways traceable to primary study or the evidences that we have synthesized. Can you include some TI numbers or another form of evidence tracking and traceability?

Excellent point—this will significantly strengthen the rigor and make reviewers happy. For IEEE Software, the best balance is to keep bullets clean while adding **lightweight traceability using Pi references** (not too dense, just representative evidence).

Here is a revised version with **traceable evidence grounding**:

## 7 KEY TAKEAWAYS FOR PRACTITIONERS

The evidence synthesized across 24 studies shows that AI-generated code delivers both benefits and risks, with outcomes highly dependent on context, task, and usage practices. The following takeaways summarize actionable guidance grounded in the reviewed evidence:

- **Always validate AI-generated code.**  
AI outputs may appear correct while containing logical or security defects (e.g., P5, P12, P16, P23). Systematic validation—through testing, code review, and static analysis—should be treated as mandatory.
- **Invest in prompt quality.**  
Prompt formulation strongly influences correctness, security, and maintainability (e.g., P11, P14, P20). Clear, structured, and context-rich prompts consistently lead to higher-quality outputs.
- **Use AI for low-risk and well-defined tasks first.**  
AI performs more reliably on routine and clearly specified problems, while error rates increase with task complexity and novelty (e.g., P3, P7, P13, P22).
- **Combine AI with testing and analysis tools.**  
Integrating AI-generated code into existing quality assurance pipelines helps detect hidden defects and vulnerabilities (e.g., P5, P9, P21, P23).
- **Avoid over-reliance on AI suggestions.**  
Over-trust and reduced scrutiny can lead to unnoticed defects and insecure code (e.g., P12, P16). Developers should remain actively engaged in evaluating outputs.
- **Iterate and refine rather than accept first outputs.**  
Iterative prompting and examining multiple generated solutions improve correctness and robustness (e.g., P13, P21, P22).

- **Be aware of variability and non-determinism.**

The same input can produce different outputs across runs, affecting reliability and reproducibility (e.g., P1, P19, P24). For critical tasks, multiple generations and consistency checks are advisable.

💡 **Key message:** AI-assisted coding is most effective when treated as a **human-AI collaborative process**, where human expertise guides, validates, and refines AI-generated outputs.

## Why this version works (for your paper)

- ✓ Keeps **IEEE Software readability**
- ✓ Adds **traceability without clutter**
- ✓ Uses **representative Pi citations (not exhaustive)**
- ✓ Aligns tightly with Sections 4–6 (reviewers will see consistency)

If you want one more refinement: we can **map each bullet explicitly to your factor/issue categories (Section 4–5–6)**—that would make the contribution even more “structured SLR-like.”



give 8 LOOKING AHEAD

Short (~10%)

Include:

- emerging benchmarks (balanced tone)
- need for better practices
- importance of human-AI collaboration

Here is a concise, IEEE Software-style **Section 8: Looking Ahead**, aligned with your paper and grounded in your synthesis:

## 8 LOOKING AHEAD

The rapid adoption of AI-assisted coding has outpaced the development of robust evaluation standards and practices. While recent studies have begun to introduce benchmarks for assessing correctness, security, and other quality attributes (e.g., P5, P24), current evaluation approaches remain fragmented and often limited to specific tasks or controlled settings. There is a growing need for more **comprehensive and realistic benchmarks** that reflect real-world development scenarios, including complex systems, evolving requirements, and long-term maintainability.

At the same time, the evidence highlights the importance of establishing **better development practices** for AI-assisted coding. This includes not only improved prompting strategies and validation workflows, but also clearer guidelines for when and how AI tools should be used. As organizations increasingly integrate AI into development pipelines, systematic approaches to quality assurance, security validation, and tool governance will become essential.

Looking forward, the most promising direction is not fully automated coding, but **effective human-AI collaboration**. The studies reviewed consistently show that the best outcomes emerge when human expertise complements AI capabilities—guiding inputs, critically evaluating outputs, and iteratively refining solutions. Strengthening this collaboration through better tool support, transparency, and developer training will be key to realizing the benefits of AI while managing its risks.

Ultimately, advancing the quality of AI-generated code will require progress on both fronts: **more reliable AI systems and more effective human practices**.



give Abstract:

Keywords: separate by ,

Here is a concise, IEEE Software-style **Abstract** and **Keywords** aligned with your paper:

## Abstract

AI-based code generation tools are increasingly integrated into software development, offering significant productivity gains but raising concerns about code quality. This article synthesizes evidence from a systematic review of 24 empirical studies to understand what influences the quality of AI-generated code and what quality issues arise in practice. The findings show that code quality is shaped by multiple interacting factors, including prompt formulation, task complexity, developer involvement, model capability, and input context. Across studies, AI-generated code exhibits substantial variability: it can produce correct and useful solutions for well-defined tasks, but also introduces risks such as logical errors, security vulnerabilities, maintainability issues, and non-deterministic behavior. We further show that defects emerge from a combination of human-related

inputs, AI limitations, and interaction misalignment between them. Overall, the evidence suggests that AI-assisted coding should be viewed as a socio-technical process, where effective human–AI collaboration and systematic validation are essential for achieving reliable outcomes. The article concludes with actionable recommendations for practitioners and directions for future research.

## Keywords

AI-generated code, code quality, large language models, GitHub Copilot, ChatGPT, software quality, empirical studies, systematic literature review, human–AI collaboration



give 100-word Abstract

Here is a ~100-word IEEE Software-style abstract (tight and balanced):

## Abstract

AI-based code generation tools are increasingly used in software development, offering productivity gains but raising concerns about code quality. This article synthesizes evidence from 24 empirical studies to examine what influences AI-generated code quality and what issues arise. The findings show that quality depends on multiple interacting factors, including prompts, task complexity, developer involvement, and model capability. While AI can produce correct solutions for well-defined tasks, it also introduces risks such as logical errors, security vulnerabilities, maintainability issues, and non-determinism. Overall, AI-assisted coding is best understood as a socio-technical process requiring careful human oversight, validation, and effective collaboration practices.

Fri, Mar 27 at 5:12 AM



we have this insight "💡 Insight: Unlike traditional programming, where identical inputs produce identical outputs, AI-generated code introduces inherent variability into the development process." . my co-author said: I am not sure if I understand this.. AI code-generation tools are yes non-deterministic but once the code is generated, it is not different that a human-written code.. I think I understnad what it is meant here, but we have to make it clear. refvis the phrasw  
Show moreShow less

Your co-author is right—the current phrasing can be misunderstood. The key is to clearly distinguish **generation-time variability** from **execution-time determinism**.

Here are a few refined options (pick the tone you prefer):

### Option 1 (clear and precise – recommended)

💡 **Insight:** Unlike traditional programming—where identical inputs produce identical outputs—AI code generation is **non-deterministic at generation time**: the same prompt can yield different code. However, once generated, the code behaves deterministically like any human-written program.

### Option 2 (slightly more formal, IEEE tone)

💡 **Insight:** While traditional programs produce identical outputs for identical inputs, AI-based code generation introduces variability at the generation stage: the same prompt may produce different implementations. Once generated, however, the code executes deterministically as in conventional software.

### Option 3 (shorter, if space is tight)

💡 **Insight:** AI code generation is non-deterministic in producing code (same prompt, different outputs), but the generated code itself executes deterministically like conventional programs.

If you want, I can also align this wording with your Section 5.4 so everything is perfectly consistent.



rethink and regenerate the insight from sractch based on that subsection : 5.4 INCONSISTENCY AND NON-DETERMINISM  
Inconsistency is examined in 12 out of 24 studies and is a distinctive characteristic of AI-generated code. The same prompt can produce different outputs across runs, varying levels of correctness, and inconsistent coding styles or structures (e.g., P4, P11, P18).  
This non-deterministic behavior makes it difficult to rely on AI outputs as stable solutions and complicates debugging and reproducibility (e.g.,

P7, P23). Unlike traditional programming, where identical inputs produce identical outputs, AI-generated code introduces inherent variability into the development process.

At the same time, recent research has begun to address this challenge. Efforts are being made to better understand and reduce non-determinism in AI-based code generation, including techniques for improving output stability and reproducibility (e.g., [7]). These approaches highlight that non-determinism is not only a limitation but also an active area of research and improvement.

[Show more](#)[Show less](#)

Great—this is the right moment to sharpen the insight so it *exactly reflects the subsection* and avoids the misunderstanding.

Here is a **rethought, precise insight** aligned with your Section 5.4:

 **Insight:** Variability in AI-assisted development arises not from how code executes, but from how it is generated: the same prompt can produce different implementations across runs, leading to inconsistent quality, behavior, and style. This generation-time non-determinism makes AI outputs less predictable and harder to reproduce than traditional development workflows.

### Why this works better

- Clarifies **generation vs execution**
- Directly ties to **inconsistency + reproducibility**
- Avoids misleading comparison with traditional programming
- Keeps **IEEE Software clarity and punch**

If you want an even tighter version:

 “AI introduces variability at code generation time: the same prompt can yield different implementations, making outputs less predictable and harder to reproduce.”



I still do not like it. say along these lines: a human developer may have a largely consistent / DETERMINISMtic coding style and approach, but when an AI model help in coding, it can have inconsistent coding style / approach. and what issue this will cause?

Ahh—now this is much clearer conceptually. You’re shifting the comparison from *execution determinism* to **consistency of coding approach/style**, which fits much better with your subsection.

Here’s a clean, IEEE Software–ready insight built on that idea:

 **Insight:** Human developers typically follow a relatively consistent coding style and problem-solving approach, whereas AI-generated code can vary across runs—even for the same task—leading to inconsistencies in structure, style, and implementation choices. This variability can reduce codebase uniformity, complicate maintenance, and make debugging and integration more challenging.

### Slightly sharper version (if you want more punch):

 **Insight:** Unlike human developers, who tend to apply consistent coding styles and approaches, AI-generated code can vary across runs for the same task. This inconsistency can reduce codebase uniformity and make maintenance, debugging, and integration more difficult.

This version:

- Matches your subsection perfectly
- Avoids the determinism confusion
- Clearly states the **practical consequence** (maintenance, debugging, integration)

If you want, I can also align this wording with your **maintainability subsection (5.3)** so the narrative becomes even tighter.